

Project Report On MedScribe AI

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Vinay Kuthey	EN22CS3011084
2	Viraj Agrawal	EN22CS3011089
3	Lakshita Singh	EN22CS3011135
4	Vishakha Shadija	EN22ME304114
5	Vishakha Hilsayan	EN22EL301065

Group Name: 09D10

Project Number: AAI-22

Industry Mentor Name:

University Mentor Name: Prof. Maya Baniya

Academic Year: 2026

Table of Contents

1. Problem Statement & Objectives

- 1.1 Introduction
- 1.2 Problem Statement
- 1.3 Project Objectives
- 1.4 Scope of the Project

2. Proposed Solution

- 2.1 Key features
- 2.2 Overall Architecture / Workflow
- 2.3 Tools & Technologies Used

3. Results & Output

- 3.1 Screenshots / outputs
- 3.2 Key outcomes

4. Conclusion

5. Future Scope & Enhancements

1. Problem Statement & Objectives

1.1 Introduction

The MedScribe AI is an Agentic-GenAI-based application designed to generate structured medical documents such as patient summaries and medical report and AI Agent that autonomously plans, validates resources, builds schedules, and replans when resources are unavailable. The system uses Retrieval-Augmented Generation (RAG) by combining a vector database (ChromaDB in-memory) with a Large Language Model (Gemini 2.5 Flash API) and LangChain integration for agentic capabilities. It ensures consistent formatting, professional tone, and domain-specific terminology by implementing instruction-based prompt engineering to generate precise response.

1.2 Problem Statement

Build a specialized GenAI content generation tool tailored for Healthcare professionals. By implementing sophisticated Prompt Engineering techniques, the application enables users to transform basic topics into high-quality professional outputs, such as 'Patient Summaries'. The system ensures consistency in tone, formatting, and industry-standard terminology, significantly reducing the manual effort required for drafting complex documentation. Integrate it with Agentic capabilities such as 'Planner Agent' designed to orchestrate complex Healthcare tasks through a multi-step reasoning loop. Upon receiving a high-level goal (e.g., 'Treatment Options'), the agent autonomously decomposes the objective into actionable steps, validates resource availability through mock interface tools, and generates a detailed execution schedule. This implementation highlights the agent's ability to handle dependencies and optimize task sequences in a dynamic environment.

1.3 Project Objectives

- Automate the generation of structured clinical documents from minimal input.
- Apply Prompt Engineering to enforce medical terminology, tone, and formatting.
- Implement Retrieval-Augmented Generation (RAG) using ChromaDB as the Vector DB.
- Provide a clean, form-based web interface for healthcare professionals.
- Using Google Gemini 2.5 Flash (free tier) as the underlying LLM.
- Make the RAG process transparent to the user through a status indicator showing when past cases were retrieved and used.
- Build a modular LangChain AI Agent with AgentExecutor, @tool functions, LLMChain, and PromptTemplate for healthcare goal planning.

1.4 Scope of the Project

MedScribe AI is demonstrating the practical integration of four AI engineering components - Prompt Engineering, Vector DB with RAG, LLM API, and LangChain in a healthcare documentation and procedure management context.

- Minimal Interface: A simple and minimal interface to enter input data and generate desired output.
- Backend Logic: Gemini 2.5 Flash with ChromaDB im-memory providing top 2 similar past cases and injected into prompt.
- LangChain Logic: LLM with LangChain integration provides agentic capabilities and provides steps schedule card for provided healthcare goal.
- Document types Supported:
 - Patient Summary — SOAP-format structured note (Subjective, Objective, Assessment, Plan)
 - Medical Report — Formal detailed diagnostic report with clinical findings and prognosis.
 - Schedule: A step-by-step schedule card with resource validation and availability check.

2. Proposed Solution

2.1 Key Features

- Clean web-based UI.
- Structured patient input form with 7 dedicated fields ensuring complete and well-organised data collection.
- Two document type options — Patient Summary (SOAP format) and Medical Report.
- Role-based Prompt Engineering templates that enforce clinical terminology, formatting, and professional tone.
- RAG architecture using ChromaDB — every generated document is embedded and stored for future context retrieval.
- Dynamic few-shot prompting — past similar cases are automatically retrieved and injected into the prompt.
- Real-time RAG status indicator showing whether Vector DB context was used, with expandable context preview.
- Zero-cost deployment using free-tier Gemini API, open-source ChromaDB
- A dedicated button to toggle to agent.
- Autonomous tool-based reasoning (ReAct Agent) with Real-time agent workflow.

2.2 Overall Architecture/ Workflow

The system follows a layered architecture consisting of four main layers: Frontend Layer, Backend Layer, Agent Layer and AI & Data Layer. It implements a Retrieval-Augmented Generation (RAG) pipeline to generate accurate and structured healthcare content.

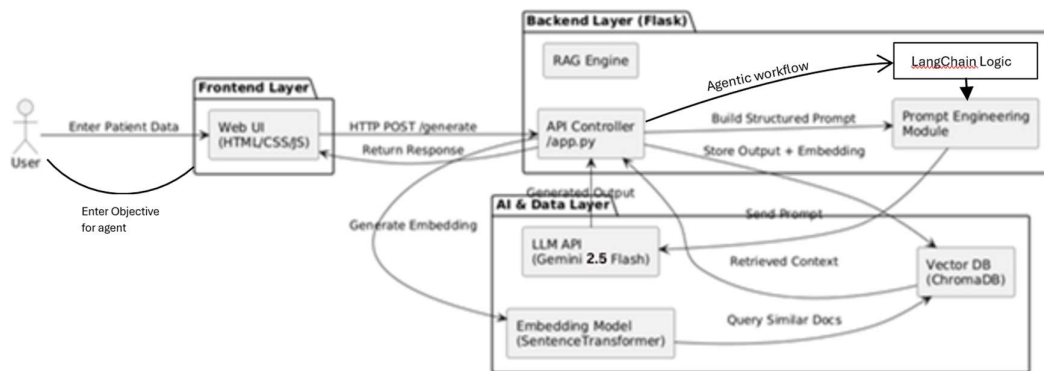


Fig. Architecture Diagram

Frontend Layer (User Interface)

- Built using HTML, CSS, and JavaScript
- Allows users (healthcare professionals) to input patient details
- Sends data to backend via HTTP POST (/generate)
- Dedicated button to switch to agent and use it for task like decomposing objective into actionable steps and also ensure resource validity.
- Displays the generated patient summary and step schedule card

Backend Layer (Flask)

- Acts as the core processing unit
- Handles API requests through the API Controller (app.py)
- Manages the RAG pipeline workflow
- Coordinates between embedding model, vector database, and LLM
- Uses a Prompt Engineering Module to:
 - Structure output in SOAP format
 - Maintain medical terminology and consistency

Agent Layer

- Provides goal to agent and then validates resource and reschedule tasks and steps if needed via langchain.
- **LangChain:**
 - AgentExecutor autonomously selects which @tool to call based on step description — no manual tool routing
 - Provides structured step-by-step procedure/ schedule card to be followed.

AI & Data Layer

This layer provides intelligence and context-aware generation:

- **Embedding Model (SentenceTransformer):**
 - Converts input text into vector embeddings
- **Vector Database (ChromaDB):**
 - Stores embeddings and generated documents
 - Retrieves similar past records for context (RAG)
- **LLM API (Gemini 2.5 Flash):**
 - Generates structured patient summaries
 - Uses both user input and retrieved context

Information Flow

For document generation

- Input Data: Patient case description or healthcare goal
- Processed Data: Embeddings (vector format)
- Context Data: Retrieved similar documents
- LLM Chain and @tools: Breaking down of steps and resource Validation
- Output Data: Structured patient summary (SOAP format)

For Agentic workflow

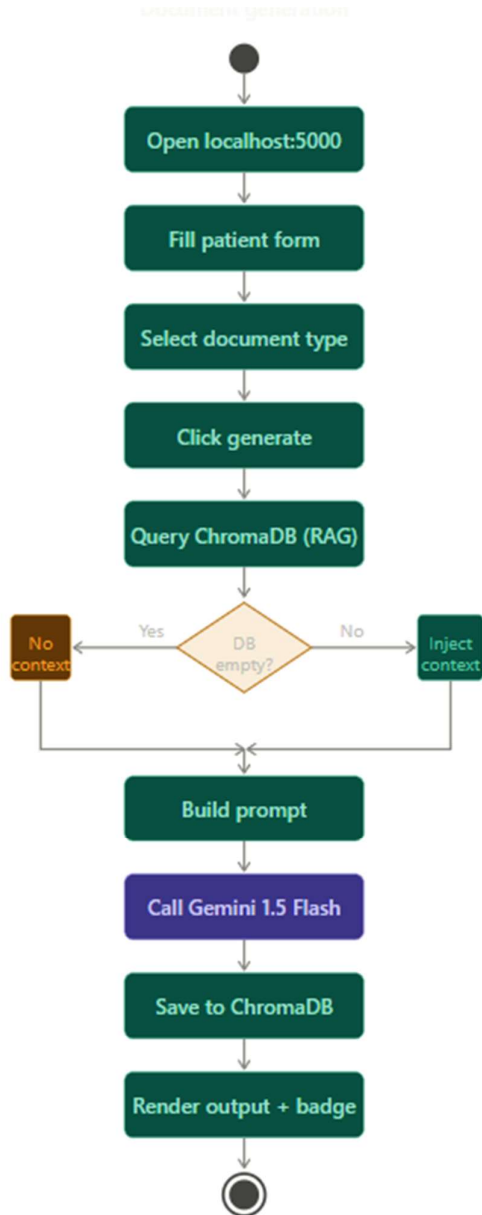
- Input Data: Healthcare goal as a prompt
- LLM Chain and @tools: Breaking down of steps and resource Validation
- Scheduling: Steps are generated and replan if needed and then finalize
- Output Data: Structured Schedule card

Flow:

User Input → Embedding → Vector DB → Context → Prompt → LLM → Output →
Storage

Activity Workflow

For Document Generation



For Agentic Workflow

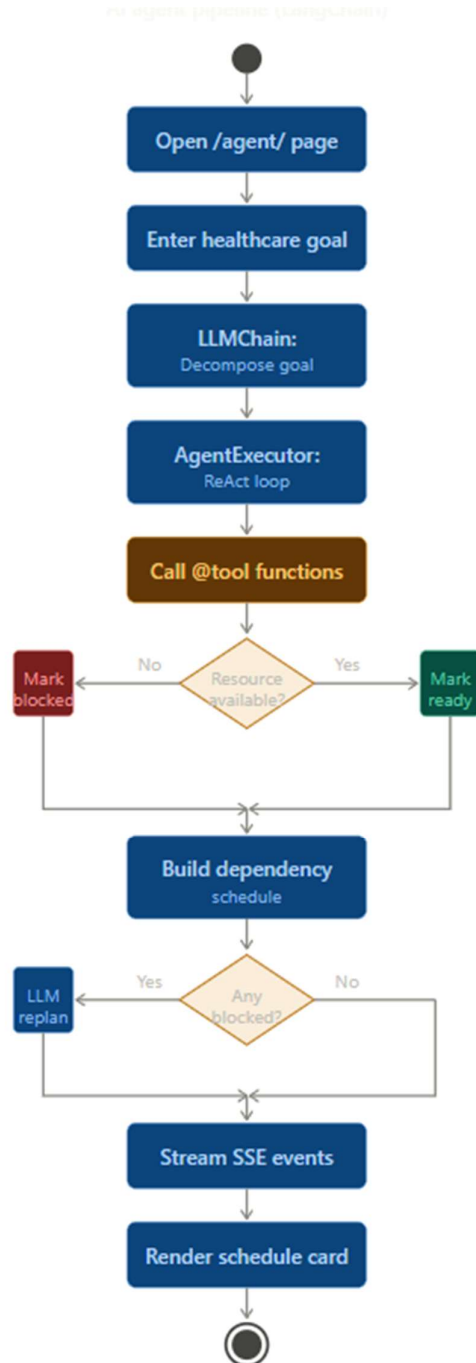


Fig. Activity Diagram

Workflow Description (Document Generation):**User Input:**

The process starts when the user enters patient details in the frontend.

Send to Backend:

The input is sent to the Flask backend via an API request.

Embedding Generation:

The backend converts the input text into vector embeddings using the embedding model.

Vector DB Query:

The system queries the vector database to find similar past documents.

Decision – Context Found?:

- **Yes:** Relevant documents are retrieved as context
- **No:** Retrieval step is skipped

Prompt Construction:

A structured prompt is created using:

- User input
- Retrieved context (if available)
- Predefined rules (SOAP format, medical terminology)

LLM Processing:

The prompt is sent to the Gemini LLM for content generation.

Output Generation:

The LLM generates a structured patient summary.

Store Output:

The generated output is stored in the vector database for future retrieval.

Return Response:

The backend sends the response back to the frontend and is displayed to the user.

Flow:

User Input → Embedding → Vector DB → Context → Prompt → LLM → Output →
Storage

Workflow Description (Agent):

User Input: The process starts when the user enters goal in the frontend.

Send to Backend: The input is sent to the Flask backend via an API request.

LLM Chain: The goal is then decomposed into small actionable steps. Also the prompt is part done here.

AgentExecutor and validation: The ReAct Loop is activated and then the tools are validated with generated schedule.

Decision – Resource Availability?:

- Yes: Mark step ready and build dependency schedule
- No: Show steps blocked and build dependency schedule

Decision – Blocked Steps?:

- Yes: Server-Sent-Events to push real time data to frontend
- No: Replan blocked steps and then SSE to frontend

Output Generation: The Final Schedule card is shown at frontend. 2.3

Flow:

User Input → LLM Chain → Tools → Steps → Finalize → Output

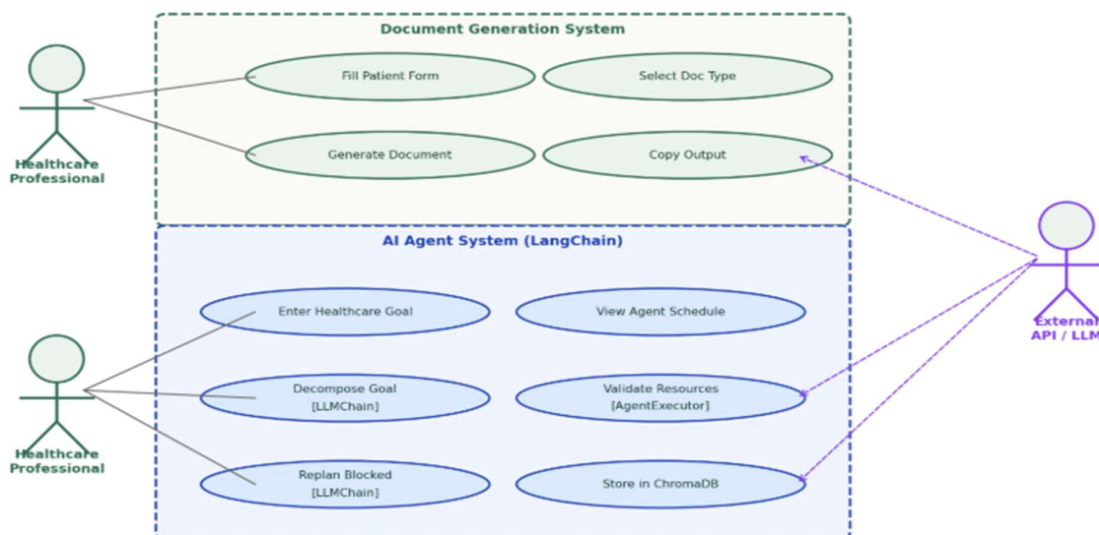


Fig. Use Case Diagram

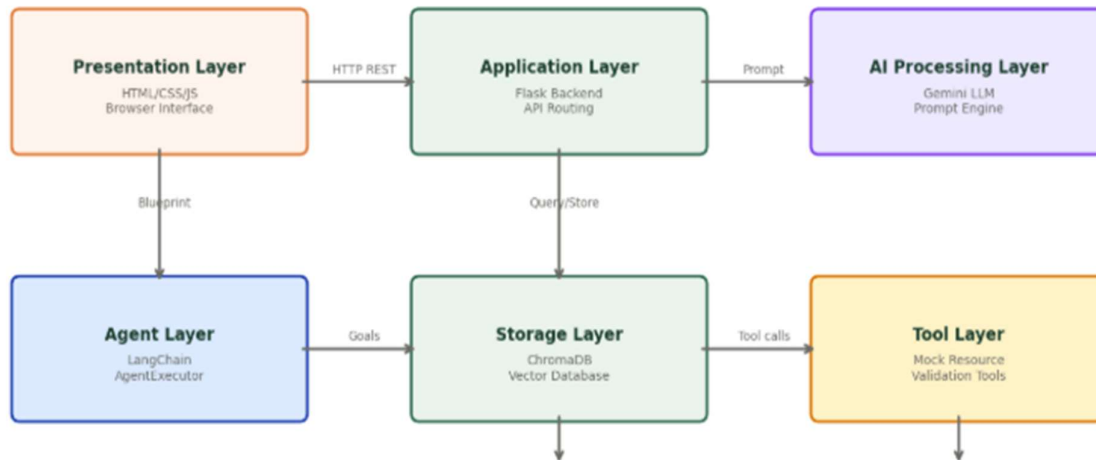


Fig. Component Diagram

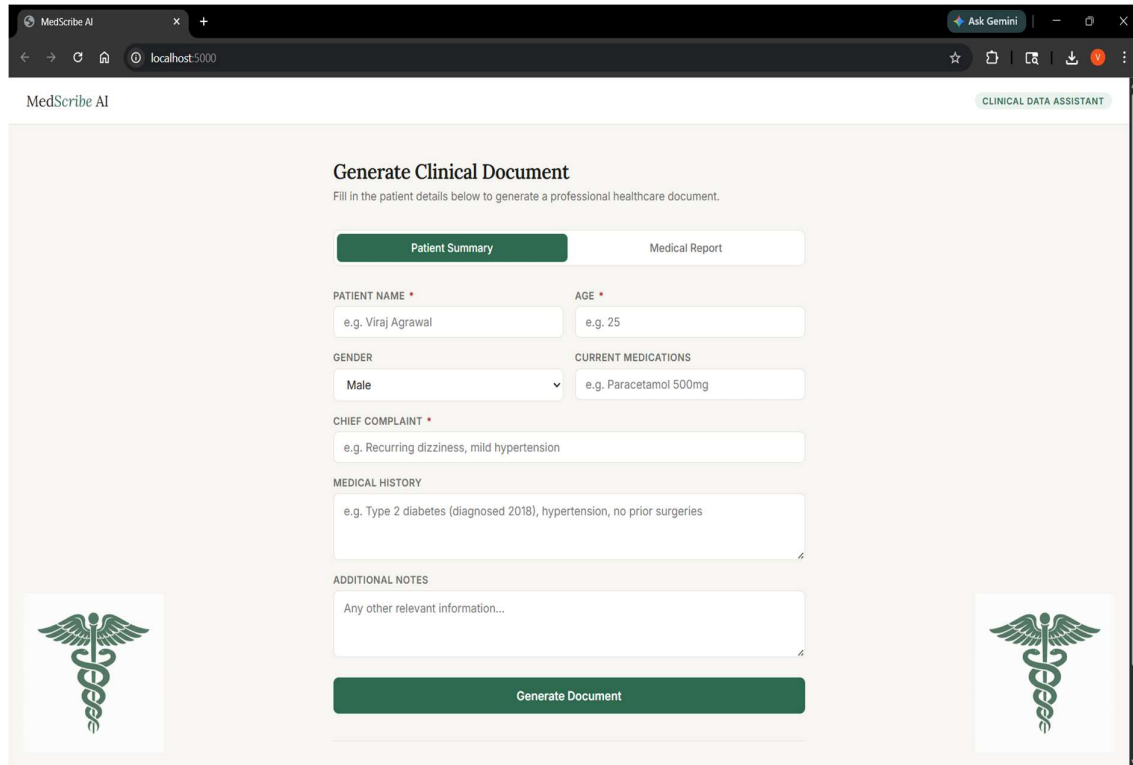
The Presentation Layer handles all browser based user interaction. The Application Layer (Flask) handles routing and orchestration. The Agent Layer implements the LangChain-powered planning pipeline and helps to provide the agentic capabilities. The Data and AI Layer contains ChromaDB, Gemini API, and LangChain component instances. The project follows a modular architecture.

2.3 Tools & Technologies

Component	Technology
Backend	Python, Flask
Frontend	HTML, CSS, JavaScript
LLM API	Google Gemini 1.5 Flash
Vector Database	ChromaDB
Agentic Framework	LangChain
Agent Components	AgentExecutor, LLMChain, PromptTemplate
Embedding	Custom SimpleEmbedder
Communication	Server-Sent Events (SSE)

3. Result & Output

3.1 Screenshots



MedScribe AI

CLINICAL DATA ASSISTANT

Generate Clinical Document

Fill in the patient details below to generate a professional healthcare document.

Patient Summary

Medical Report

PATIENT NAME *

AGE *

GENDER

CURRENT MEDICATIONS

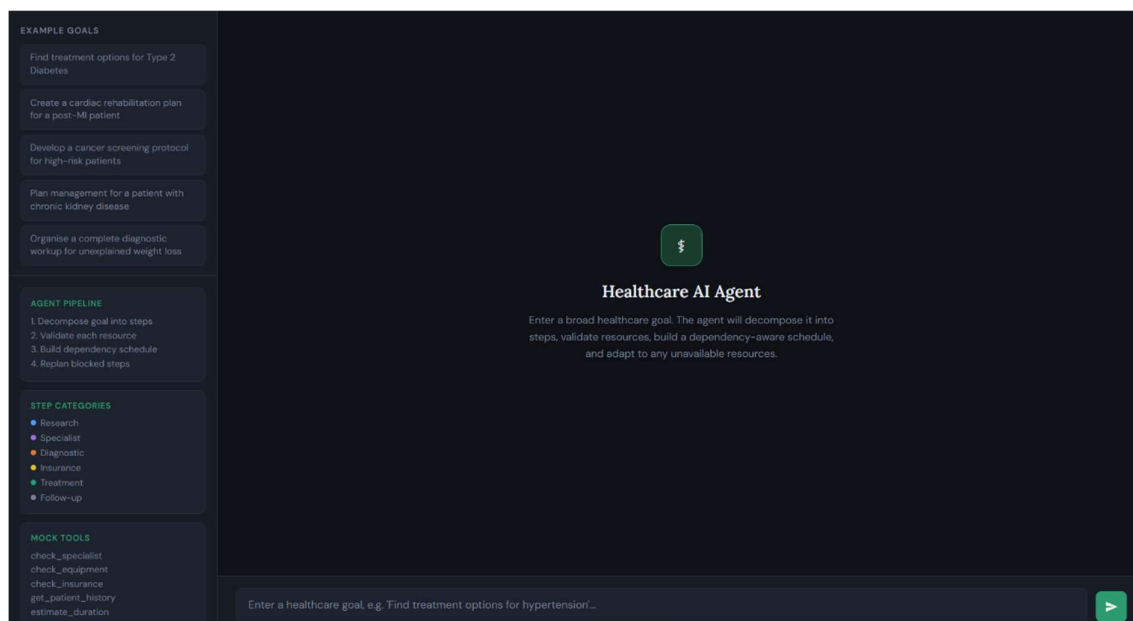
CHIEF COMPLAINT *

MEDICAL HISTORY

ADDITIONAL NOTES

Generate Document

Homepage



EXAMPLE GOALS

- Find treatment options for Type 2 Diabetes
- Create a cardiac rehabilitation plan for a post-MI patient
- Develop a cancer screening protocol for high-risk patients
- Plan management for a patient with chronic kidney disease
- Organise a complete diagnostic workup for unexplained weight loss

AGENT PIPELINE

- Decompose goal into steps
- Validate each resource
- Build dependency-aware schedule
- Replan blocked steps

STEP CATEGORIES

- Research
- Specialist
- Diagnostic
- Insurance
- Treatment
- Follow-up

MOCK TOOLS

- check_specialist
- check_equipment
- check_insurance
- get_patient_history
- estimate_duration

Healthcare AI Agent

Enter a broad healthcare goal. The agent will decompose it into steps, validate resources, build a dependency-aware schedule, and adapt to any unavailable resources.

Enter a healthcare goal, e.g. Find treatment options for hypertension...

Agent Interface

3.2 Key Outcomes

- Automated clinical documentation generation
- Improved output quality using RAG
- Successful implementation of autonomous AI agent
- Modular architecture enabling easy scalability
- Real-time AI interaction via streaming

4. Conclusion

MedScribe AI successfully demonstrates a practical two-tier GenAI architecture for healthcare: a document generation system using Prompt Engineering and RAG, and an autonomous planning agent built on LangChain. The key design achievement is the complete modularity — the agent feature adds significant AI capability through LangChain (AgentExecutor, LLMChain, PromptTemplate, @tool) while requiring minimal change to the existing, working codebase and files.

The use of LangChain and its importance is shown effectively. LangChain streamlines AI development by offering built-in components to format prompts, connect models, create AI-accessible tools, and empower agents to autonomously choose how to complete a task.

5. Future Scope & Enhancements

- Switch ChromaDB to PersistentClient("./db") for permanent RAG memory across server restarts.
- Replace SimpleEmbedder with BioBERT or ClinicalBERT for semantically accurate medical embeddings.
- Implement PDF/Word export of generated documents directly from the UI.
- Add voice input using the Web Speech API for hands-free patient data entry.
- Integrate with EHR systems via FHIR API using a LangChain custom tool.
- Apply formal RAGAs evaluation framework to measure Faithfulness, Context Precision, and Answer Relevance.
- Conduct human evaluation by licensed clinicians on generated document quality